School of Future Tech

Case Study Report

on

Picture-In-Picture overlay workspace utility

by

Pallavi sarovar

150096725219

Index

---

1. Introduction to the Case Study

Modern productivity workflows require users to continuously switch between multiple browser tabs, applications, and workspaces. Important information such as notes, timers, and media streams often become hidden behind other windows, leading to reduced focus and efficiency.

This case study focuses on the design and implementation of a **Picture-In-Picture Overlay Workspace Utility**, a frontend-only ReactJS application that enables users to detach workspace modules into floating always-on-top windows using the browser's native **Document Picture-in-Picture API**.

The application demonstrates how modern browser technologies can be combined to create a productivity-focused workspace environment. Users can interact with notes, stopwatches, and media streams while maintaining visibility of these tools regardless of the active application.

The project highlights the practical use of browser APIs, state management, workspace persistence, and real-time synchronization in a modern web application.

---

2. Problem Statement / Case Background (Abstract)

Background

Traditional productivity tools operate entirely within a browser tab or application window. When users switch to another application, important information such as notes, timers, or media feeds becomes inaccessible without switching back.

Modern browsers now provide support for the **Document Picture-in-Picture API**, which allows arbitrary HTML content to be displayed in floating windows that remain visible above other applications.

This capability creates opportunities for building productivity-oriented workspace utilities that support detachable widgets and real-time synchronization.

Abstract

This case study presents the design and implementation of a **Picture-In-Picture Overlay Workspace Utility** using ReactJS, Zustand, Tailwind CSS, and modern browser APIs.

The application enables users to detach workspace modules such as a Markdown Notepad, Focus Stopwatch, and Universal Media Stream Viewer into independent floating windows. State synchronization between the main application and detached windows is managed through Zustand, while LocalStorage provides session persistence.

The implementation also includes telemetry monitoring, media stream management, workspace export and restore functionality, and browser capability detection. The project demonstrates how frontend technologies and browser APIs can be combined to create advanced productivity applications without requiring any backend infrastructure.

---

3. Case Study Design

The Picture-In-Picture Overlay Workspace Utility is designed as a modular frontend application consisting of the following stages:

1. Workspace Selection

Users select a workspace module from the dashboard.

Available modules include:

- Notes Workspace
- Stopwatch Workspace
- Media Stream Viewer
- Telemetry Console

2. State Management

All application data is managed using a centralized Zustand store.

Managed state includes:

- Active workspace
- Notes content
- Checklist items
- Stopwatch information
- PiP status
- Media information
- Telemetry logs

3. Picture-In-Picture Integration

Users can detach supported modules into floating overlay windows.

The process includes:

- Creating a PiP window
- Cloning stylesheets
- Rendering React components
- Maintaining synchronization

4. Persistence Layer

Workspace data is automatically stored in LocalStorage.

Stored data includes:

- Notes
- Checklist items
- Preferences
- Workspace settings

5. Import and Export System

Users can:

- Export workspace data as JSON
- Restore workspace configurations

- Recover previous sessions

6. Telemetry Monitoring

The application continuously tracks:

- System events
- Media events
- Workspace events
- PiP lifecycle events

---

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key Methods and Technologies

1. React Component Architecture

The application follows a component-based architecture where each workspace module is implemented as an independent React component.

Examples include:

- FloatingNotesWidget
- StopwatchWidget
- ScreenShareWidget
- TelemetryConsole

2. State Management

The Zustand library is used for:

- Global state management
- Real-time synchronization
- Cross-window updates

3. Document Picture-in-Picture API

The browser's Document Picture-in-Picture API is used to:

- Create floating windows
- Render React components
- Support detached workspaces

4. Local Storage Persistence

LocalStorage is used to:

- Save notes
- Store preferences
- Preserve workspace settings

5. Media Stream Management

MediaDevices API and Screen Capture API are used for:

- Webcam streaming
- Screen sharing
- Video management

Technology Stack Used for the Case

Programming Language

- JavaScript

Frameworks and Libraries

- ReactJS
- Vite
- Zustand
- Tailwind CSS

Browser APIs

- Document Picture-in-Picture API
- Picture-in-Picture API
- MediaDevices API
- Screen Capture API
- LocalStorage API

Deployment Platform

- Vercel

---

5. Problem Statement / Case Study Implementation Details and Snapshots

The implementation is organized into multiple modules and utility files.

1. workspaceStore.js

Responsible for:

- Global state management
- Synchronization
- Persistence handling

2. usePictureInPicture.js

Responsible for:

- PiP lifecycle management
- Window creation
- Style cloning
- Event handling

3. FloatingNotesWidget.jsx

Provides:

- Note editing
- Checklist support
- Synchronization

4. StopwatchWidget.jsx

Provides:

- Timer controls
- Session tracking
- PiP compatibility

5. ScreenShareWidget.jsx

Provides:

- Webcam support
- Screen sharing
- Video upload support

6. TelemetryConsole.jsx

Provides:

- Event logging
- Activity tracking
- System monitoring

Snapshots Included

Include the following screenshots in the report:

- Dashboard Interface
- Notes Workspace
- Stopwatch Workspace
- Media Stream Viewer
- Picture-In-Picture Window
- Telemetry Console
- Export / Restore Functionality
- Browser Capability Dashboard

---

6. Problem Statement / Case Study Results and Conclusion

Findings

The implementation successfully demonstrates the practical use of the Document Picture-in-Picture API for productivity-focused applications.

Key observations include:

- Detached widgets remain synchronized with the main application.
- LocalStorage effectively preserves user data between sessions.
- Media streaming functionality integrates successfully with floating windows.
- Zustand simplifies cross-window state management.

- The frontend-only architecture eliminates backend complexity.

Conclusion

This case study successfully implements a complete **Picture-In-Picture Overlay Workspace Utility** using ReactJS and modern browser technologies.

The project integrates workspace management, media streaming, telemetry monitoring, state synchronization, and floating overlay windows into a unified productivity platform.

Through the use of ReactJS, Zustand, Tailwind CSS, LocalStorage, and the Document Picture-in-Picture API, the application demonstrates how advanced browser features can be leveraged to improve multitasking and productivity while maintaining a lightweight frontend-only architecture.

---

7. References

- ReactJS Official Documentation – https://react.dev
- Vite Official Documentation – https://vitejs.dev
- Zustand Documentation – https://zustand-demo.pmnd.rs
- Tailwind CSS Documentation – https://tailwindcss.com
- MDN Web Docs – MediaDevices API
- MDN Web Docs – Screen Capture API
- Google Chrome Documentation – Document Picture-in-Picture API
- JavaScript LocalStorage Documentation
- Modern Frontend Development Resources and Tutorials